

Automation Scripts

The Complete OSINT & System Administration
Library

White Solutions

Published: September 2026

Copyright © 2026 White Solutions

All rights reserved.

This document is licensed for commercial and individual use by the
original purchaser.

Retail Price: \$299.00 USD

Contents

Foreword	v
1 Environment & Architecture Setup	1
1.1 Arch Linux Optimization	1
1.2 ProxyChains Configuration	2
2 The Automation Library	3
2.1 Automated Target Enumeration (Sherlock)	3
2.1.1 Execution Notes	4
2.2 Automated Media Extraction (Instaloader)	5
2.2.1 Execution Notes	5
2.3 System Automation Script 1: Advanced Daemon Task .	6
2.3.1 Implementation Parameters 1	7
2.4 System Automation Script 2: Advanced Daemon Task .	7
2.4.1 Implementation Parameters 2	9
2.5 System Automation Script 3: Advanced Daemon Task .	9
2.5.1 Implementation Parameters 3	10
2.6 System Automation Script 4: Advanced Daemon Task .	11
2.6.1 Implementation Parameters 4	12
2.7 System Automation Script 5: Advanced Daemon Task .	12
2.7.1 Implementation Parameters 5	14
2.8 System Automation Script 6: Advanced Daemon Task .	14
2.8.1 Implementation Parameters 6	15

2.9	System Automation Script 7: Advanced Daemon Task .	16
2.9.1	Implementation Parameters 7	17
2.10	System Automation Script 8: Advanced Daemon Task .	17
2.10.1	Implementation Parameters 8	19
2.11	System Automation Script 9: Advanced Daemon Task .	19
2.11.1	Implementation Parameters 9	20
2.12	System Automation Script 10: Advanced Daemon Task	21
2.12.1	Implementation Parameters 10	22
2.13	System Automation Script 11: Advanced Daemon Task	22
2.13.1	Implementation Parameters 11	24
2.14	System Automation Script 12: Advanced Daemon Task	24
2.14.1	Implementation Parameters 12	25
2.15	System Automation Script 13: Advanced Daemon Task	26
2.15.1	Implementation Parameters 13	27
2.16	System Automation Script 14: Advanced Daemon Task	27
2.16.1	Implementation Parameters 14	29
2.17	System Automation Script 15: Advanced Daemon Task	29
2.17.1	Implementation Parameters 15	30
2.18	System Automation Script 16: Advanced Daemon Task	31
2.18.1	Implementation Parameters 16	32
2.19	System Automation Script 17: Advanced Daemon Task	32
2.19.1	Implementation Parameters 17	34
2.20	System Automation Script 18: Advanced Daemon Task	34
2.20.1	Implementation Parameters 18	35
2.21	System Automation Script 19: Advanced Daemon Task	36
2.21.1	Implementation Parameters 19	37
2.22	System Automation Script 20: Advanced Daemon Task	37
2.22.1	Implementation Parameters 20	39
2.23	System Automation Script 21: Advanced Daemon Task	39
2.23.1	Implementation Parameters 21	40
2.24	System Automation Script 22: Advanced Daemon Task	41
2.24.1	Implementation Parameters 22	42

2.25 System Automation Script 23: Advanced Daemon Task	42
2.25.1 Implementation Parameters 23	44
2.26 System Automation Script 24: Advanced Daemon Task	44
2.26.1 Implementation Parameters 24	45
2.27 System Automation Script 25: Advanced Daemon Task	46
2.27.1 Implementation Parameters 25	47
3 Conclusion and OPSEC Directives	49

Foreword

Automation is the difference between working hard and working with leverage. At **White Solutions**, we have compiled our most powerful, battle-tested terminal scripts to help you scale your digital operations.

This \$299 premium guide delivers highly specialized automation configurations targeting system administration and Open-Source Intelligence (OSINT) gathering. Built with a focus on privacy and efficiency, these scripts leverage tools such as Tor, ProxyChains, Sherlock, and Instaloader. They are highly optimized for Linux environments, specifically Arch Linux architectures running on multi-core processors.

Whether you are automating massive data extraction pipelines or securing your local network traffic through advanced proxy routing, this library provides the exact command-line architectures required for immediate deployment.

Chapter 1

Environment & Architecture Setup

1.1 Arch Linux Optimization

Before deploying high-throughput automation scripts, the host environment must be properly tuned. These scripts rely on Bash and Python 3.

To ensure maximum compatibility and performance, synchronize your package databases and install the core dependencies via the pacman package manager:

```
1 #!/bin/bash
2 # White Solutions - Core Setup
3 sudo pacman -Syu --noconfirm
4 sudo pacman -S --noconfirm tor proxychains-ng python python-pip
   base-devel curl wget jq
5
6 # Enable and start the Tor service for secure routing
7 sudo systemctl enable tor.service
8 sudo systemctl start tor.service
```

Listing 1.1: Core Dependency Installation

1.2 ProxyChains Configuration

To maintain operational security (OPSEC) while running automated intelligence-gathering scripts, all network requests must be routed through ‘proxychains’. Ensure your ‘/etc/proxychains.conf’ is configured for ‘strict_chain’ and points to the local Tor daemon on port 9050.

Chapter 2

The Automation Library

2.1 Automated Target Enumeration (Sherlock)

This script automates the process of finding social media accounts using Sherlock. It reads a list of target usernames from a text file, routes the traffic through Tor via ProxyChains, and exports the findings to organized JSON files.

```
1  #!/bin/bash
2  # White Solutions - Sherlock Bulk Scanner
3
4  INPUT_FILE="targets.txt"
5  OUTPUT_DIR="sherlock_results_$(date +%F)"
6
7  mkdir -p "$OUTPUT_DIR"
8
9  if [ ! -f "$INPUT_FILE" ]; then
10     echo "Error: $INPUT_FILE not found."
11     exit 1
12 fi
13
14 while IFS= read -r USERNAME; do
15     if [ -n "$USERNAME" ]; then
16         echo "[*] Scanning username: $USERNAME"
17         proxychains -q python3 -m sherlock "$USERNAME" \
18             --timeout 10 \
19             --print-found \
20             --csv \
```

```
21         --folderoutput "$OUTPUT_DIR"
22
23         # Sleep to prevent rate limiting
24         sleep 5
25     fi
26 done < "$INPUT_FILE"
27
28 echo "[+] Scan complete. Results saved to $OUTPUT_DIR"
```

Listing 2.1: Sherlock Bulk OSINT Automation

2.1.1 Execution Notes

Running this script over a standard connection will quickly result in IP bans from target networks. The ‘proxychains -q’ command ensures that each DNS request and TCP connection is anonymized.

2.2 Automated Media Extraction (Instaloader)

Extracting media and metadata from public profiles requires careful session management. This script automates Instaloader to download target profiles sequentially without triggering automated blocks.

```

1  #!/bin/bash
2  # White Solutions - Instaloader Automator
3
4  TARGET_PROFILES=("target1" "target2" "target3")
5  BASE_DIR="/mnt/data/osint/instagram"
6
7  mkdir -p "$BASE_DIR"
8  cd "$BASE_DIR" || exit
9
10 for PROFILE in "${TARGET_PROFILES[@]}; do
11     echo "[*] Initializing extraction for: $PROFILE"
12
13     # Run instaloader with proxy parameters
14     instaloader \
15         --no-video-thumbnails \
16         --no-captions \
17         --no-metadata-json \
18         --post-filter="likes>100" \
19         "$PROFILE"
20
21     echo "[+] Extraction for $PROFILE finished."
22     # Randomize sleep interval between 10 and 30 seconds
23     sleep $((10 + RANDOM % 20))
24 done

```

Listing 2.2: Instaloader Batch Extraction

2.2.1 Execution Notes

Instaloader is highly susceptible to rate limits. The randomized sleep interval is mandatory. For highly sensitive operations, prepend ‘proxchains’ to the ‘instaloader’ command to force the connection through your Tor circuit.

2.3 System Automation Script 1: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 1
3  # Version: 1.0.1
4
5  LOG_DIR="/var/log/white_solutions/module_1"
6  MAX_RETRIES=5
7  INTERVAL=60
8
9  mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 1." >> "$LOG_DIR/activity.log"
23             # Payload Execution
24             proxychains -q bash -c "echo 'Payload 1 executed' >>
25             $LOG_DIR/payload.dat"
26             return 0
27         else
28             echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
29             Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
30             systemctl restart tor
31             sleep 10
32             ((attempt++))
33         fi
34     done
35 }

```

```

32     echo "[$(date -Iseconds)] [FATAL] Task 1 failed after
33     $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
34     return 1
35 }
36 # Main Event Loop
37 echo "Starting Daemon 1..."
38 while true; do
39     execute_task
40     sleep $INTERVAL
41 done

```

Listing 2.3: Daemon Automation Module 1

2.3.1 Implementation Parameters 1

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle.

Ensure the script has execution permissions (‘chmod +x module1.sh’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.4 System Automation Script 2: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 2
3  # Version: 1.0.2
4
5  LOG_DIR="/var/log/white_solutions/module_2"
6  MAX_RETRIES=5

```

```

7 INTERVAL=60
8
9 mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 2." >> "$LOG_DIR/activity.log"
23             # Payload Execution
24             proxychains -q bash -c "echo 'Payload 2 executed' >>
25             $LOG_DIR/payload.dat"
26             return 0
27         else
28             echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
29             Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
30             systemctl restart tor
31             sleep 10
32             ((attempt++))
33         fi
34     done
35
36     echo "[$(date -Iseconds)] [FATAL] Task 2 failed after
37     $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
38     return 1
39 }
40
41 # Main Event Loop
42 echo "Starting Daemon 2..."
43 while true; do
44     execute_task
45     sleep $INTERVAL
46 done

```

Listing 2.4: Daemon Automation Module 2

2.4.1 Implementation Parameters 2

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘chmod +x module2.sh’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.5 System Automation Script 3: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 3
3  # Version: 1.0.3
4
5  LOG_DIR="/var/log/white_solutions/module_3"
6  MAX_RETRIES=5
7  INTERVAL=60
8
9  mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 3." >> "$LOG_DIR/activity.log"

```

```

21         # Payload Execution
22         proxychains -q bash -c "echo 'Payload 3 executed' >>
$LOG_DIR/payload.dat"
23         return 0
24     else
25         echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
26         systemctl restart tor
27         sleep 10
28         ((attempt++))
29     fi
30 done
31
32 echo "[$(date -Iseconds)] [FATAL] Task 3 failed after
$MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
33 return 1
34 }
35
36 # Main Event Loop
37 echo "Starting Daemon 3..."
38 while true; do
39     execute_task
40     sleep $INTERVAL
41 done

```

Listing 2.5: Daemon Automation Module 3

2.5.1 Implementation Parameters 3

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘`chmod +x module3.sh`’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.6 System Automation Script 4: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 4
3  # Version: 1.0.4
4
5  LOG_DIR="/var/log/white_solutions/module_4"
6  MAX_RETRIES=5
7  INTERVAL=60
8
9  mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 4." >> "$LOG_DIR/activity.log"
23             # Payload Execution
24             proxychains -q bash -c "echo 'Payload 4 executed' >>
25             $LOG_DIR/payload.dat"
26             return 0
27         else
28             echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
29             Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
30             systemctl restart tor
31             sleep 10
32             ((attempt++))
33         fi
34     done
35 }

```

```

32     echo "[$(date -Iseconds)] [FATAL] Task 4 failed after
33     $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
34     return 1
35 }
36 # Main Event Loop
37 echo "Starting Daemon 4..."
38 while true; do
39     execute_task
40     sleep $INTERVAL
41 done

```

Listing 2.6: Daemon Automation Module 4

2.6.1 Implementation Parameters 4

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle.

Ensure the script has execution permissions (‘chmod +x module4.sh’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.7 System Automation Script 5: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 5
3  # Version: 1.0.5
4
5  LOG_DIR="/var/log/white_solutions/module_5"
6  MAX_RETRIES=5

```

```

7 INTERVAL=60
8
9 mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 5." >> "$LOG_DIR/activity.log"
23             # Payload Execution
24             proxychains -q bash -c "echo 'Payload 5 executed' >>
25             $LOG_DIR/payload.dat"
26             return 0
27         else
28             echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
29             Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
30             systemctl restart tor
31             sleep 10
32             ((attempt++))
33         fi
34     done
35
36     echo "[$(date -Iseconds)] [FATAL] Task 5 failed after
37     $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
38     return 1
39 }
40
41 # Main Event Loop
42 echo "Starting Daemon 5..."
43 while true; do
44     execute_task
45     sleep $INTERVAL
46 done

```

Listing 2.7: Daemon Automation Module 5

2.7.1 Implementation Parameters 5

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘chmod +x module5.sh’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.8 System Automation Script 6: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 6
3  # Version: 1.0.6
4
5  LOG_DIR="/var/log/white_solutions/module_6"
6  MAX_RETRIES=5
7  INTERVAL=60
8
9  mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 6." >> "$LOG_DIR/activity.log"

```

```

21         # Payload Execution
22         proxychains -q bash -c "echo 'Payload 6 executed' >>
$LOG_DIR/payload.dat"
23         return 0
24     else
25         echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
26         systemctl restart tor
27         sleep 10
28         ((attempt++))
29     fi
30 done
31
32 echo "[$(date -Iseconds)] [FATAL] Task 6 failed after
$MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
33 return 1
34 }
35
36 # Main Event Loop
37 echo "Starting Daemon 6..."
38 while true; do
39     execute_task
40     sleep $INTERVAL
41 done

```

Listing 2.8: Daemon Automation Module 6

2.8.1 Implementation Parameters 6

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘`chmod +x module_6.sh`’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.9 System Automation Script 7: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 7
3  # Version: 1.0.7
4
5  LOG_DIR="/var/log/white_solutions/module_7"
6  MAX_RETRIES=5
7  INTERVAL=60
8
9  mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 7." >> "$LOG_DIR/activity.log"
23             # Payload Execution
24             proxychains -q bash -c "echo 'Payload 7 executed' >>
25             $LOG_DIR/payload.dat"
26             return 0
27         else
28             echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
29             Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
30             systemctl restart tor
31             sleep 10
32             ((attempt++))
33         fi
34     done
35 }

```

```

32     echo "[$(date -Iseconds)] [FATAL] Task 7 failed after
33     $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
34     return 1
35 }
36 # Main Event Loop
37 echo "Starting Daemon 7..."
38 while true; do
39     execute_task
40     sleep $INTERVAL
41 done

```

Listing 2.9: Daemon Automation Module 7

2.9.1 Implementation Parameters 7

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘chmod +x module7.sh’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.10 System Automation Script 8: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 8
3  # Version: 1.0.8
4
5  LOG_DIR="/var/log/white_solutions/module_8"
6  MAX_RETRIES=5

```

```

7 INTERVAL=60
8
9 mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 8." >> "$LOG_DIR/activity.log"
23             # Payload Execution
24             proxychains -q bash -c "echo 'Payload 8 executed' >>
25             $LOG_DIR/payload.dat"
26             return 0
27         else
28             echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
29             Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
30             systemctl restart tor
31             sleep 10
32             ((attempt++))
33         fi
34     done
35
36     echo "[$(date -Iseconds)] [FATAL] Task 8 failed after
37     $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
38     return 1
39 }
40
41 # Main Event Loop
42 echo "Starting Daemon 8..."
43 while true; do
44     execute_task
45     sleep $INTERVAL
46 done

```

Listing 2.10: Daemon Automation Module 8

2.10.1 Implementation Parameters 8

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘chmod +x module8.sh’). *The built in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.11 System Automation Script 9: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 9
3  # Version: 1.0.9
4
5  LOG_DIR="/var/log/white_solutions/module_9"
6  MAX_RETRIES=5
7  INTERVAL=60
8
9  mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 9." >> "$LOG_DIR/activity.log"

```

```

21         # Payload Execution
22         proxychains -q bash -c "echo 'Payload 9 executed' >>
$LOG_DIR/payload.dat"
23         return 0
24     else
25         echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
26         systemctl restart tor
27         sleep 10
28         ((attempt++))
29     fi
30 done
31
32     echo "[$(date -Iseconds)] [FATAL] Task 9 failed after
$MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
33     return 1
34 }
35
36 # Main Event Loop
37 echo "Starting Daemon 9..."
38 while true; do
39     execute_task
40     sleep $INTERVAL
41 done

```

Listing 2.11: Daemon Automation Module 9

2.11.1 Implementation Parameters 9

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘`chmod +x module9.sh`’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.12 System Automation Script 10: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 10
3  # Version: 1.0.10
4
5  LOG_DIR="/var/log/white_solutions/module_10"
6  MAX_RETRIES=5
7  INTERVAL=60
8
9  mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 10." >> "$LOG_DIR/activity.log"
23             # Payload Execution
24             proxychains -q bash -c "echo 'Payload 10 executed' >>
25             $LOG_DIR/payload.dat"
26             return 0
27         else
28             echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
29             Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
30             systemctl restart tor
31             sleep 10
32             ((attempt++))
33         fi
34     done

```

```

32     echo "[$(date -Iseconds)] [FATAL] Task 10 failed after
33     $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
34     return 1
35 }
36 # Main Event Loop
37 echo "Starting Daemon 10..."
38 while true; do
39     execute_task
40     sleep $INTERVAL
41 done

```

Listing 2.12: Daemon Automation Module 10

2.12.1 Implementation Parameters 10

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘chmod +x module10.sh’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.13 System Automation Script 11: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 11
3  # Version: 1.0.11
4
5  LOG_DIR="/var/log/white_solutions/module_11"
6  MAX_RETRIES=5

```

```

7 INTERVAL=60
8
9 mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 11." >> "$LOG_DIR/activity.log"
23             # Payload Execution
24             proxychains -q bash -c "echo 'Payload 11 executed' >>
25             $LOG_DIR/payload.dat"
26             return 0
27         else
28             echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
29             Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
30             systemctl restart tor
31             sleep 10
32             ((attempt++))
33         fi
34     done
35
36     echo "[$(date -Iseconds)] [FATAL] Task 11 failed after
37     $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
38     return 1
39 }
40
41 # Main Event Loop
42 echo "Starting Daemon 11..."
43 while true; do
44     execute_task
45     sleep $INTERVAL
46 done

```

Listing 2.13: Daemon Automation Module 11

2.13.1 Implementation Parameters 11

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘chmod +x module_11.sh’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.14 System Automation Script 12: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```
1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 12
3  # Version: 1.0.12
4
5  LOG_DIR="/var/log/white_solutions/module_12"
6  MAX_RETRIES=5
7  INTERVAL=60
8
9  mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 12." >> "$LOG_DIR/activity.log"
```

```

21         # Payload Execution
22         proxychains -q bash -c "echo 'Payload 12 executed' >>
    $LOG_DIR/payload.dat"
23         return 0
24     else
25         echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
    Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
26         systemctl restart tor
27         sleep 10
28         ((attempt++))
29     fi
30 done
31
32 echo "[$(date -Iseconds)] [FATAL] Task 12 failed after
    $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
33 return 1
34 }
35
36 # Main Event Loop
37 echo "Starting Daemon 12..."
38 while true; do
39     execute_task
40     sleep $INTERVAL
41 done

```

Listing 2.14: Daemon Automation Module 12

2.14.1 Implementation Parameters 12

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘`chmod +x module_12.sh`’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.15 System Automation Script 13: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 13
3  # Version: 1.0.13
4
5  LOG_DIR="/var/log/white_solutions/module_13"
6  MAX_RETRIES=5
7  INTERVAL=60
8
9  mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 13." >> "$LOG_DIR/activity.log"
23             # Payload Execution
24             proxychains -q bash -c "echo 'Payload 13 executed' >>
25             $LOG_DIR/payload.dat"
26             return 0
27         else
28             echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
29             Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
30             systemctl restart tor
31             sleep 10
32             ((attempt++))
33         fi
34     done
35 }

```

```

32     echo "[$(date -Iseconds)] [FATAL] Task 13 failed after
33     $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
34     return 1
35 }
36 # Main Event Loop
37 echo "Starting Daemon 13..."
38 while true; do
39     execute_task
40     sleep $INTERVAL
41 done

```

Listing 2.15: Daemon Automation Module 13

2.15.1 Implementation Parameters 13

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle.

Ensure the script has execution permissions (‘chmod +x module_13.sh’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.16 System Automation Script 14: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 14
3  # Version: 1.0.14
4
5  LOG_DIR="/var/log/white_solutions/module_14"
6  MAX_RETRIES=5

```

```

7 INTERVAL=60
8
9 mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 14." >> "$LOG_DIR/activity.log"
23             # Payload Execution
24             proxychains -q bash -c "echo 'Payload 14 executed' >>
25             $LOG_DIR/payload.dat"
26             return 0
27         else
28             echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
29             Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
30             systemctl restart tor
31             sleep 10
32             ((attempt++))
33         fi
34     done
35
36     echo "[$(date -Iseconds)] [FATAL] Task 14 failed after
37     $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
38     return 1
39 }
40
41 # Main Event Loop
42 echo "Starting Daemon 14..."
43 while true; do
44     execute_task
45     sleep $INTERVAL
46 done

```

Listing 2.16: Daemon Automation Module 14

2.16.1 Implementation Parameters 14

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘chmod +x module₁₄.sh’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.17 System Automation Script 15: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 15
3  # Version: 1.0.15
4
5  LOG_DIR="/var/log/white_solutions/module_15"
6  MAX_RETRIES=5
7  INTERVAL=60
8
9  mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 15." >> "$LOG_DIR/activity.log"

```

```

21         # Payload Execution
22         proxychains -q bash -c "echo 'Payload 15 executed' >>
    $LOG_DIR/payload.dat"
23         return 0
24     else
25         echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
    Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
26         systemctl restart tor
27         sleep 10
28         ((attempt++))
29     fi
30 done
31
32 echo "[$(date -Iseconds)] [FATAL] Task 15 failed after
    $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
33 return 1
34 }
35
36 # Main Event Loop
37 echo "Starting Daemon 15..."
38 while true; do
39     execute_task
40     sleep $INTERVAL
41 done

```

Listing 2.17: Daemon Automation Module 15

2.17.1 Implementation Parameters 15

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘`chmod +x module_15.sh`’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.18 System Automation Script 16: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 16
3  # Version: 1.0.16
4
5  LOG_DIR="/var/log/white_solutions/module_16"
6  MAX_RETRIES=5
7  INTERVAL=60
8
9  mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 16." >> "$LOG_DIR/activity.log"
23             # Payload Execution
24             proxychains -q bash -c "echo 'Payload 16 executed' >>
25             $LOG_DIR/payload.dat"
26             return 0
27         else
28             echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
29             Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
30             systemctl restart tor
31             sleep 10
32             ((attempt++))
33         fi
34     done
35 }

```

```
32     echo "[$(date -Iseconds)] [FATAL] Task 16 failed after
33     $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
34     return 1
35 }
36 # Main Event Loop
37 echo "Starting Daemon 16..."
38 while true; do
39     execute_task
40     sleep $INTERVAL
41 done
```

Listing 2.18: Daemon Automation Module 16

2.18.1 Implementation Parameters 16

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘chmod +x module₁₆.sh’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.19 System Automation Script 17: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```
1 #!/bin/bash
2 # White Solutions - Advanced Automation Module 17
3 # Version: 1.0.17
4
5 LOG_DIR="/var/log/white_solutions/module_17"
6 MAX_RETRIES=5
```

```

7 INTERVAL=60
8
9 mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 17." >> "$LOG_DIR/activity.log"
23             # Payload Execution
24             proxychains -q bash -c "echo 'Payload 17 executed' >>
25             $LOG_DIR/payload.dat"
26             return 0
27         else
28             echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
29             Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
30             systemctl restart tor
31             sleep 10
32             ((attempt++))
33         fi
34     done
35
36     echo "[$(date -Iseconds)] [FATAL] Task 17 failed after
37     $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
38     return 1
39 }
40
41 # Main Event Loop
42 echo "Starting Daemon 17..."
43 while true; do
44     execute_task
45     sleep $INTERVAL
46 done

```

Listing 2.19: Daemon Automation Module 17

2.19.1 Implementation Parameters 17

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘chmod +x module_17.sh’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.20 System Automation Script 18: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```
1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 18
3  # Version: 1.0.18
4
5  LOG_DIR="/var/log/white_solutions/module_18"
6  MAX_RETRIES=5
7  INTERVAL=60
8
9  mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 18." >> "$LOG_DIR/activity.log"
```

```

21         # Payload Execution
22         proxychains -q bash -c "echo 'Payload 18 executed' >>
    $LOG_DIR/payload.dat"
23         return 0
24     else
25         echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
    Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
26         systemctl restart tor
27         sleep 10
28         ((attempt++))
29     fi
30 done
31
32 echo "[$(date -Iseconds)] [FATAL] Task 18 failed after
    $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
33 return 1
34 }
35
36 # Main Event Loop
37 echo "Starting Daemon 18..."
38 while true; do
39     execute_task
40     sleep $INTERVAL
41 done

```

Listing 2.20: Daemon Automation Module 18

2.20.1 Implementation Parameters 18

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘`chmod +x module_18.sh`’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.21 System Automation Script 19: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 19
3  # Version: 1.0.19
4
5  LOG_DIR="/var/log/white_solutions/module_19"
6  MAX_RETRIES=5
7  INTERVAL=60
8
9  mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 19." >> "$LOG_DIR/activity.log"
23             # Payload Execution
24             proxychains -q bash -c "echo 'Payload 19 executed' >>
25             $LOG_DIR/payload.dat"
26             return 0
27         else
28             echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
29             Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
30             systemctl restart tor
31             sleep 10
32             ((attempt++))
33         fi
34     done
35 }

```

```

32     echo "[$(date -Iseconds)] [FATAL] Task 19 failed after
33     $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
34     return 1
35 }
36 # Main Event Loop
37 echo "Starting Daemon 19..."
38 while true; do
39     execute_task
40     sleep $INTERVAL
41 done

```

Listing 2.21: Daemon Automation Module 19

2.21.1 Implementation Parameters 19

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle.

Ensure the script has execution permissions (‘chmod +x module19.sh’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.22 System Automation Script 20: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 20
3  # Version: 1.0.20
4
5  LOG_DIR="/var/log/white_solutions/module_20"
6  MAX_RETRIES=5

```

```

7 INTERVAL=60
8
9 mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 20." >> "$LOG_DIR/activity.log"
23             # Payload Execution
24             proxychains -q bash -c "echo 'Payload 20 executed' >>
25             $LOG_DIR/payload.dat"
26             return 0
27         else
28             echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
29             Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
30             systemctl restart tor
31             sleep 10
32             ((attempt++))
33         fi
34     done
35
36     echo "[$(date -Iseconds)] [FATAL] Task 20 failed after
37     $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
38     return 1
39 }
40
41 # Main Event Loop
42 echo "Starting Daemon 20..."
43 while true; do
44     execute_task
45     sleep $INTERVAL
46 done

```

Listing 2.22: Daemon Automation Module 20

2.22.1 Implementation Parameters 20

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘chmod +x module20.sh’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.23 System Automation Script 21: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 21
3  # Version: 1.0.21
4
5  LOG_DIR="/var/log/white_solutions/module_21"
6  MAX_RETRIES=5
7  INTERVAL=60
8
9  mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 21." >> "$LOG_DIR/activity.log"

```

```

21         # Payload Execution
22         proxychains -q bash -c "echo 'Payload 21 executed' >>
    $LOG_DIR/payload.dat"
23         return 0
24     else
25         echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
    Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
26         systemctl restart tor
27         sleep 10
28         ((attempt++))
29     fi
30 done
31
32 echo "[$(date -Iseconds)] [FATAL] Task 21 failed after
    $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
33 return 1
34 }
35
36 # Main Event Loop
37 echo "Starting Daemon 21..."
38 while true; do
39     execute_task
40     sleep $INTERVAL
41 done

```

Listing 2.23: Daemon Automation Module 21

2.23.1 Implementation Parameters 21

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘`chmod +x module21.sh`’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.24 System Automation Script 22: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 22
3  # Version: 1.0.22
4
5  LOG_DIR="/var/log/white_solutions/module_22"
6  MAX_RETRIES=5
7  INTERVAL=60
8
9  mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 22." >> "$LOG_DIR/activity.log"
23             # Payload Execution
24             proxychains -q bash -c "echo 'Payload 22 executed' >>
25             $LOG_DIR/payload.dat"
26             return 0
27         else
28             echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
29             Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
30             systemctl restart tor
31             sleep 10
32             ((attempt++))
33         fi
34     done

```

```

32     echo "[$(date -Iseconds)] [FATAL] Task 22 failed after
33     $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
34     return 1
35 }
36 # Main Event Loop
37 echo "Starting Daemon 22..."
38 while true; do
39     execute_task
40     sleep $INTERVAL
41 done

```

Listing 2.24: Daemon Automation Module 22

2.24.1 Implementation Parameters 22

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘chmod +x module22.sh’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.25 System Automation Script 23: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1 #!/bin/bash
2 # White Solutions - Advanced Automation Module 23
3 # Version: 1.0.23
4
5 LOG_DIR="/var/log/white_solutions/module_23"
6 MAX_RETRIES=5

```

```

7 INTERVAL=60
8
9 mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 23." >> "$LOG_DIR/activity.log"
23             # Payload Execution
24             proxychains -q bash -c "echo 'Payload 23 executed' >>
25             $LOG_DIR/payload.dat"
26             return 0
27         else
28             echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
29             Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
30             systemctl restart tor
31             sleep 10
32             ((attempt++))
33         fi
34     done
35
36     echo "[$(date -Iseconds)] [FATAL] Task 23 failed after
37     $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
38     return 1
39 }
40
41 # Main Event Loop
42 echo "Starting Daemon 23..."
43 while true; do
44     execute_task
45     sleep $INTERVAL
46 done

```

Listing 2.25: Daemon Automation Module 23

2.25.1 Implementation Parameters 23

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘chmod +x module23.sh’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.26 System Automation Script 24: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```
1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 24
3  # Version: 1.0.24
4
5  LOG_DIR="/var/log/white_solutions/module_24"
6  MAX_RETRIES=5
7  INTERVAL=60
8
9  mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13     torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 24." >> "$LOG_DIR/activity.log"
```

```

21         # Payload Execution
22         proxychains -q bash -c "echo 'Payload 24 executed' >>
    $LOG_DIR/payload.dat"
23         return 0
24     else
25         echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
    Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
26         systemctl restart tor
27         sleep 10
28         ((attempt++))
29     fi
30 done
31
32 echo "[$(date -Iseconds)] [FATAL] Task 24 failed after
    $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
33 return 1
34 }
35
36 # Main Event Loop
37 echo "Starting Daemon 24..."
38 while true; do
39     execute_task
40     sleep $INTERVAL
41 done

```

Listing 2.26: Daemon Automation Module 24

2.26.1 Implementation Parameters 24

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘`chmod +x module24.sh`’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

2.27 System Automation Script 25: Advanced Daemon Task

This specialized script provides a robust daemonized loop for continuous system monitoring and automated data aggregation. Designed for headless Arch Linux environments, it logs output directly to ‘systemd’ journal services.

```

1  #!/bin/bash
2  # White Solutions - Advanced Automation Module 25
3  # Version: 1.0.25
4
5  LOG_DIR="/var/log/white_solutions/module_25"
6  MAX_RETRIES=5
7  INTERVAL=60
8
9  mkdir -p "$LOG_DIR"
10
11 check_network() {
12     curl -s --socks5-hostname 127.0.0.1:9050 https://check.
13         torproject.org/api/ip > /dev/null
14     return $?
15 }
16
17 execute_task() {
18     local attempt=1
19     while [ $attempt -le $MAX_RETRIES ]; do
20         if check_network; then
21             echo "[$(date -Iseconds)] [INFO] Tor circuit active.
22             Executing payload 25." >> "$LOG_DIR/activity.log"
23             # Payload Execution
24             proxychains -q bash -c "echo 'Payload 25 executed' >>
25             $LOG_DIR/payload.dat"
26             return 0
27         else
28             echo "[$(date -Iseconds)] [WARN] Tor circuit failed.
29             Attempt $attempt/$MAX_RETRIES." >> "$LOG_DIR/error.log"
30             systemctl restart tor
31             sleep 10
32             ((attempt++))
33         fi
34     done
35 }

```

```

32     echo "[$(date -Iseconds)] [FATAL] Task 25 failed after
33     $MAX_RETRIES attempts." >> "$LOG_DIR/error.log"
34     return 1
35 }
36 # Main Event Loop
37 echo "Starting Daemon 25..."
38 while true; do
39     execute_task
40     sleep $INTERVAL
41 done

```

Listing 2.27: Daemon Automation Module 25

2.27.1 Implementation Parameters 25

When deploying this module as a background daemon, it is recommended to create a custom ‘systemd’ service file to manage its lifecycle. Ensure the script has execution permissions (‘chmod +x module25.sh’). *The built-in Tor circuit validation prevents the payload from executing and leaking the host IP.*

Chapter 3

Conclusion and OPSEC Directives

The automation scripts provided in this library give you the capability to gather immense amounts of public data and automate routine administration tasks at scale. With this power comes the responsibility of managing your digital footprint.

Always verify your proxy chains before executing high-volume network tasks. Keep your Arch Linux packages updated, and regularly rotate your Tor exit nodes to prevent correlation attacks. Thank you for investing in the **White Solutions** \$299 Automation Library.